FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

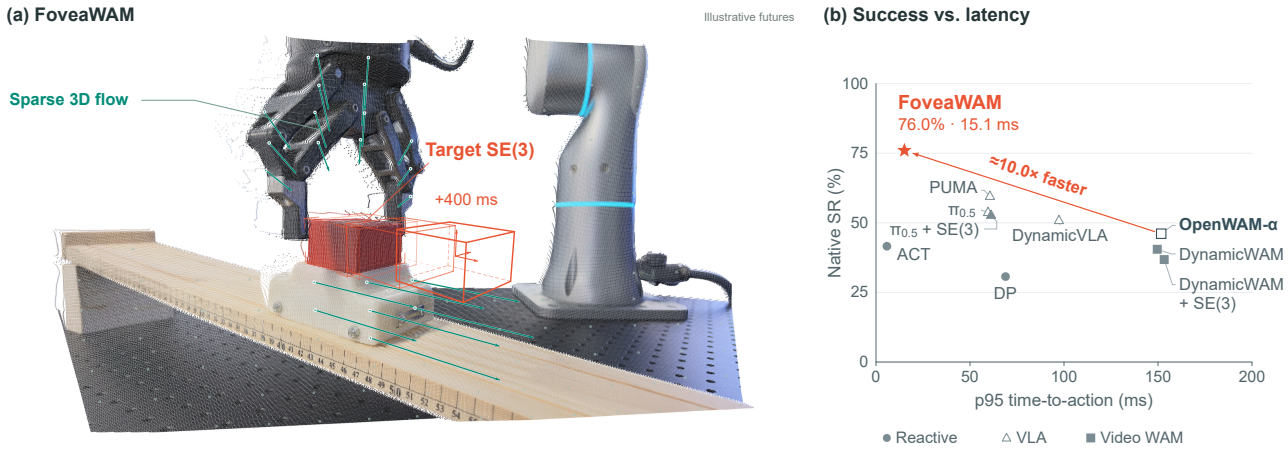


Figure 1. **FoveaWAM allocates prediction precision by task role.** (a) A real-robot scene reconstructed from RGB with MoGe-2 [42], with illustrative target SE(3) futures and sparse 3D flow. (b) Native SR versus p95 time-to-action from Table 1.

Abstract

Manipulating moving objects requires actions that fit the scene when they execute, not when the image was captured, yet every millisecond spent inferring the scene's motion lets the target move further. World–action models forecast future images or latents at uniform fidelity, spending time on appearance while leaving the target's metric pose implicit; target-centric predictors are fast but ignore the surroundings that constrain or drive the motion. We observe that the parts of a dynamic scene need different precision: the target's pose decides contact to within millimeters, the bodies around it matter only through the clearance they leave and the motion they impart to the target, and the remaining background does not matter. We present FoveaWAM, a world–action model built on this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history, so one RGB-D frame suffices as input. It learns to forecast the target's future SE(3) trajectory precisely and the whole scene coarsely as sparse 3D flow, with forecasts used only in training. FoveaWAM acts in 15.1 ms on one RTX 5090, about 10× faster than video world–action baselines. We also introduce OnTimeBench, an open-source benchmark of 14 simulated dynamic manipulation tasks in which the scene keeps moving throughout each policy's inference. FoveaWAM achieves 76.0% success on OnTimeBench, 16.4 points above the strongest baseline, also leads on 24 tasks of the DOMINO benchmark with 32.5% SR, and succeeds in [XX]% of trials on six real-robot tasks with diverse target motion.

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [5, 6, 12, 20, 43, 49, 63], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives, computes, and acts. In such scenes, an action must fit the scene at the moment it executes, not the moment its observation was captured, so a policy has to know how the scene moves. That knowledge must itself be inferred, and inference takes time during which the scene keeps moving: the longer a policy computes to anticipate the motion, the further ahead it must anticipate.

Recent dynamic policies add a memory of past motion, typically image-space cues such as optical flow [14, 27], and a forecast of the future [27, 38, 56]. Video-based WAMs forecast whole future frames or latents at uniform fidelity, spending much of their computation on visual detail [43, 49]

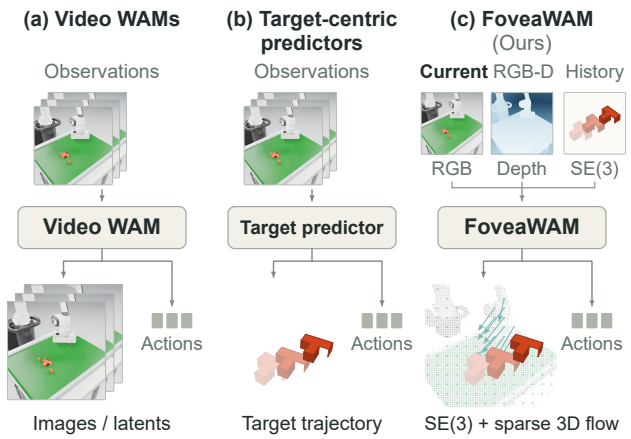


Figure 2. **What dynamic policies forecast.** (a) Video WAMs forecast future frames or latents at uniform fidelity [43, 49]. (b) Target-centric predictors forecast only the target's trajectory [14, 17]. (c) FoveaWAM reads one RGB-D frame with the target's SE(3) history and forecasts the target's SE(3) trajectory precisely and the whole scene coarsely as sparse 3D flow; both forecasts supervise training and are skipped at inference.

(Fig. 2a). Compact future codes [28] reduce this cost and metric 3D scene dynamics [54] add geometry, but both still treat the whole scene uniformly. In each case, the quantity that decides contact, the target's metric pose at execution time, stays implicit. Target-centric predictors and dynamic VLAs with object-level forecasts sit at the other end [14, 17] (Fig. 2b): they are fast but anticipate only the target, leaving out the surroundings that can block or redirect its motion. Neither approach allocates precision according to what contact requires from each part of the scene. This raises a central question: what must a policy know about a moving scene, and how precisely, to make contact while keeping inference latency low?

Our key observation is that a dynamic policy needs to know very little precisely (Fig. 1). Contact demands millimeter precision in the target's pose at execution time; the surroundings matter only through the clearance they leave and the motion they impart to the target, so they need only be known coarsely. Building on this observation, we introduce FoveaWAM, a world–action model that learns a role-aware metric 4D representation, precise where contact demands it (Fig. 2c). To supply motion information without encoding past image frames, FoveaWAM pairs a single RGB-D frame with a short history of the target's metric SE(3) poses. A world stream learns the representation by forecasting the target's future SE(3) trajectory and the motion of the whole scene as a sparse 3D flow. An action stream reads the world stream's scene and summary tokens to generate action chunks, with explicit forecasting confined to training [53]. FoveaWAM thus combines precise anticipation of target motion with low inference latency, limiting how much the scene changes between observation and action.

We build OnTimeBench, an open-source benchmark of 14 simulated dynamic manipulation tasks in which the scene keeps moving throughout each policy's inference, so every method pays for its own latency. FoveaWAM achieves 76.0% success on OnTimeBench, 16.4 points above the strongest baseline, at a p95 latency of 15.1 ms with compiled BF16 inference, about 10× lower than that of the video world–action baselines [27, 43]. It also leads on 24 DOMINO tasks [14] with 32.5% SR, 5.6 points above DynamicWAM, and succeeds in [XX]% of trials on six real-robot tasks, against [XX]% for the strongest baseline. Controlled ablations test the contributions of target-motion history and future supervision, and compare role-aware with uniform-fidelity forecasts to examine the allocation of precision.

Our contributions are threefold:

- We show that a dynamic policy needs millimeter precision only in the target's pose at execution time, and instantiate this principle as role-aware 4D modeling: a precise metric SE(3) trajectory for the target and a coarse sparse 3D flow for the whole scene.
- We develop FoveaWAM, a world–action model that combines metric target-pose histories with role-aware predictive supervision for low-latency action generation.
- We introduce OnTimeBench, an open-source dynamic manipulation benchmark that charges each policy its own inference latency, and demonstrate gains over strong baselines on OnTimeBench, 24 DOMINO tasks, and six real-robot tasks.

2. Related Work

Dynamic manipulation. Compensating for the time between sensing and acting is a classical problem, from control with dead time [35] to catching systems that track the target with Kalman-type filters and plan an interception [1–3, 30]; such systems are precise but engineered for each object and task. Learned policies have recently been brought to moving scenes [47]. PUMA reads historical optical flow and is supervised with future object features [14], and DynamicWAM conditions a WAM on flow and kinematic descriptors and executes it with real-time chunking [27]. Both infer past motion from image-space cues, whereas FoveaWAM takes the target's metric SE(3) pose history as input and observes only the current RGB-D frame. Other dynamic policies forecast salient tokens [38], future images [57], compact future codes [28], or 3D scene dynamics [54] and condition their actions on these forecasts at test time. DynaWM adapts a base VLA's action chunks for dynamic manipulation [10], and DynamicManip synthesizes dynamic demonstrations from a static one [22]. Execution schemes reduce or hide latency by overlapping inference with execution [33, 47], keeping consecutive chunks consistent [7, 25], or condition-

ing on the execution time [39]. They decide when actions are computed and applied, not what the policy represents. DOMINO [14] and PhysMani-Bench [54] provide diverse dynamic-manipulation tasks, but their released default evaluation pipelines advance the simulation only after a policy has returned its prediction. OnTimeBench advances the scene during inference and charges each method its measured latency.

Predictive supervision in WAMs. World–action models (WAMs) generate future video and actions in one network [21, 43, 49, 61], building on video prediction as a policy objective [40, 46]. Because generating video is costly, recent work changes what is forecast, such as dynamic regions, depth, and semantic features [56], flow and geometry alongside RGB [52], object masks or slots [26, 50], or compact latents [37, 60], and asks whether the forecast is needed at test time. Fast-WAM retains most of the benefit of video modeling without test-time imagination [53], and PUMA and SG-WAM drop their predictors after training [14, 58]. FoveaWAM keeps this training-only use but forecasts primarily a metric 4D state rather than pixels, features, or latents, so the target's future pose is supervised explicitly instead of being left implicit.

Object-centric and geometric motion representations. Metric 3D inputs make policies more precise and sample-efficient [32, 55]. Motion has been represented by 2D point tracks [4, 45, 48], 3D flow [15, 18, 51, 54], and object keypoints or SE(3) trajectories [17, 29, 36], while rigid-body decompositions have long been used to learn scene dynamics [8, 9]. These works describe the scene with one kind of representation, such as dense flow or tracks everywhere, or a pose for each object. Target-centric predictors, including dynamic VLAs with object-level forecasts, model only the target's future [14, 17] and leave out the bodies that block or push it. DOCIR separates the robot, the target, and obstacles, but uses these distinctions only as input encodings [13]. FoveaWAM assigns precision by role: a coarse sparse 3D flow describes the whole scene, while the target, on which contact depends to within millimeters, is additionally described by its SE(3) history and forecast.

3. FoveaWAM

We formulate dynamic manipulation as generating time-indexed command chunks (Sec. 3.1) from a representation supervised, at the execution times of these commands, with the scene's sparse 3D flow and the target's SE(3) trajectory (Fig. 3). We describe the metric inputs (Sec. 3.2), this supervision (Sec. 3.3), and how an action stream reads the representation to generate the chunk (Sec. 3.4).

3.1. Problem Formulation

We study manipulation in scenes that keep moving while the policy computes. At query t, a policy π learned from demonstrations reads an RGB image I_t and a depth map D_t captured at time τ_t, together with histories $\mathcal{T}_t$ of the target pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$ and $\mathcal{R}_t$ of the robot state, all in the robot base frame, and outputs a chunk of H commands

$$(\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1}) \sim \pi\big(\cdot \mid I_t, D_t, \mathcal{T}_t, \mathcal{R}_t\big), \qquad (1)$$

where command j is meant for time $\tau_t + j\Delta_a$, with control period Δ_a.

Each task specifies a *target*, whose pose the robot must match within a contact or containment tolerance. The surroundings that matter form the *context*: obstacles, which the robot must clear, and drivers, which move the target. The chunk becomes available only after inference, so each executed command must account for the target and the context at its execution time rather than at τ_t.

3.2. Metric Scene and Motion Inputs

FoveaWAM encodes the target's motion precisely and its context through the current geometry: the current RGB-D frame gives the metric layout of the scene, and a short history of target poses gives the motion that one frame does not show, which also reflects how drivers have moved the target.

Current scene. FoveaWAM anchors each patch token at a metric 3D point, so that the observation shares the base frame of the forecasts and the actions. A frozen MAE ViT-B/16 [16] turns the RGB image into 196 patch tokens on a 14×14 grid, and the depth map back-projects the center of patch i to a point $\mathbf{x}_i \in \mathbb{R}^3$ in the base frame. A linear embedding of $\mathbf{x}_i$ is added to the patch token, and $\mathbf{x}_i$ drives 3D rotary position embeddings of the patch queries and keys, so that attention between patches depends on their relative 3D positions. The points $\mathbf{x}_i$ are also the source points of the scene flow (Sec. 3.3).

Target motion. Instead of past frames, from which motion must be recovered by comparing images, FoveaWAM receives the target's recent motion as a metric SE(3) state. The history spans the last 350 ms in 8 samples at 20 Hz; its 8 tokens hold the current target pose and the seven motions between consecutive samples, each with its translation, relative rotation, time interval, and finite-difference velocity and acceleration in physical units. The velocity and acceleration that decide where the target will be thus enter the model directly, at the cost of a few low-dimensional tokens; Sec. 4.3 removes this history to measure its effect. In simulation, we use ground-truth target poses from the simulator. On the robot, a causal tracker estimates them from the RGB-D stream.

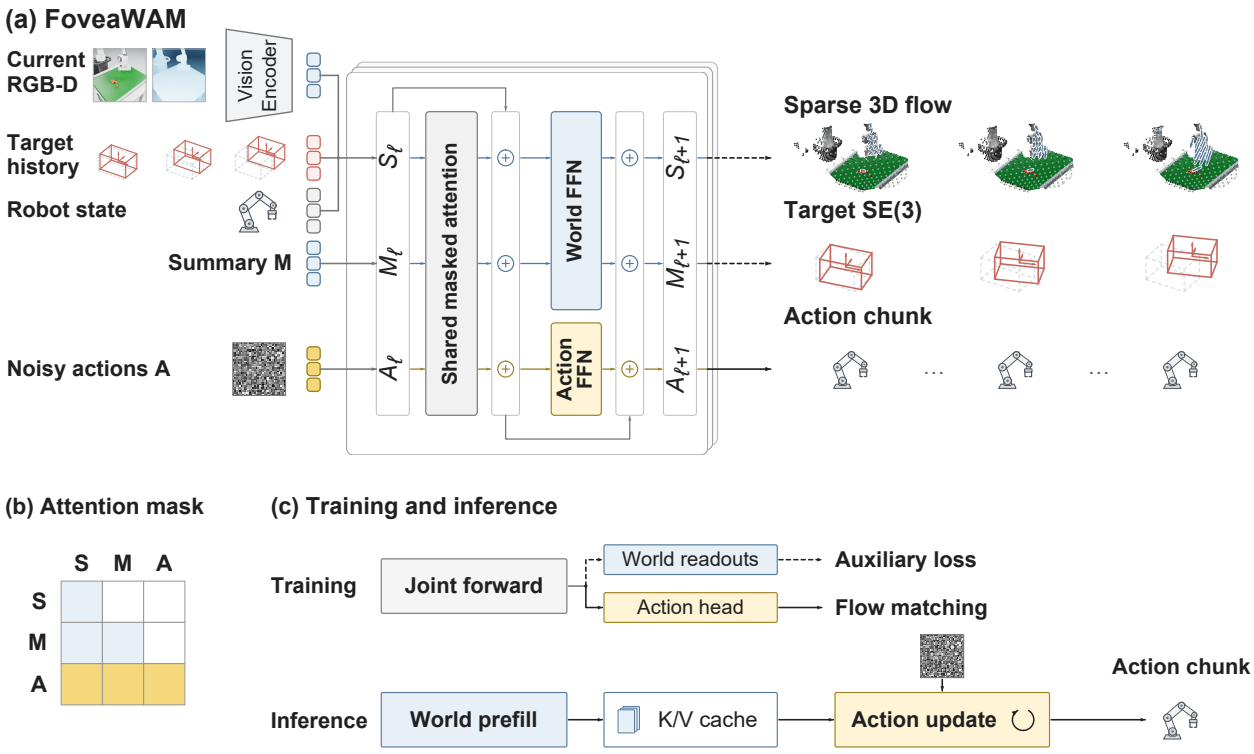


Figure 3. **FoveaWAM architecture.** (a) Current RGB-D and target/robot histories form scene tokens S, alongside learned summaries M and noisy actions A. Each layer shares masked attention, applies one World FFN to S, M and a separate Action FFN to A, and retains both residual updates. Final patch and mean-summary states support training-only geometric readouts (dashed); actions read hidden states. (b) Mask rows are queries and columns are keys. (c) Joint training combines auxiliary supervision and action flow matching. At deployment, world prefill caches layer-wise keys and values for action sampling, skipping geometric readouts. The RGB thumbnail is a Blender rendering. Geometric examples show recorded supervision (flow enlarged); token counts, layer stacking and action icons are schematic.

World stream. The patch tokens, the target tokens, and 8 robot tokens with the end-effector states at the times of the target history form the 212 scene tokens. Together with 16 learnable summary tokens that aggregate them, they make up the world stream.

3.3. Role-Aware 4D Predictive Supervision

FoveaWAM supervises its representation with two geometric forecasts of different precision: the motion of the whole scene as a sparse 3D flow on the patch grid, and, because the target's pose decides contact, the full SE(3) trajectory of the target. Both are expressed in the base frame as changes from the current state at the times $\tau_t + \delta_h$, with $\delta_h = h{\cdot}50\,\text{ms}$ for $h = 1, \ldots, 8$, from 50 ms to 400 ms after capture; together with the current state, these horizons cover the execution times of all commands in the chunk.

Scene flow. At the resolution of the patch grid, the flow covers every surface that the patches see: the robot, the target, and the context, whose clearance and motion the commands must account for. For each of the 196 points $\mathbf{x}_i$ and each horizon h, a linear head on the final state of patch token i predicts the point's base-frame displacement $\hat{F}_{ih} \in \mathbb{R}^3$ and a visibility logit $\hat{o}_{ih}$, trained with

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h)\in\mathcal{V}_F} \Big[\tfrac{1}{3} \big\| (\hat{F}_{ih} - F_{ih})/s \big\|^2 + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \tag{2}$$

where F_{ih} is the displacement of the surface point at $\mathbf{x}_i$, o_{ih} its visibility, s a fixed scale fitted on the training labels, and $\mathcal{V}_F$ the set of valid labels.

Target trajectory. The flow samples the target only at the few patches it covers, too sparse to determine its rotation, so a dedicated linear head on the mean of the final summary states forecasts its rigid motion directly: for each horizon h, the target's displacement $\hat{\mathbf{d}}_h$ and a rotation increment $\hat{\boldsymbol{\phi}}_h$,

trained with

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h \in \mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \Big], \quad (3)$$

where $\mathbf{d}_h = \mathbf{p}(\tau_t{+}\delta_h) - \mathbf{p}(\tau_t)$ and $R_h = R(\tau_t{+}\delta_h)$ are the true displacement and orientation, $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^{\wedge}) R(\tau_t)$, $d_{\mathrm{SO}(3)}$ is the geodesic distance, and $\mathcal{V}$ is the set of horizons with valid labels. The rotation term matters for contact: a rotation error θ can displace a contact point at distance r from the target's origin by up to $2r\sin(\theta/2)$. For example, a 5° error at $r = 10\,\mathrm{cm}$ can shift the contact point by nearly 9 mm.

3.4. World–Action Learning and Inference

Action stream. The action stream generates the chunk by conditional flow matching [23]. It holds $H = 8$ tokens, each embedding one noisy command: an end-effector translation increment, a rotation increment, and a gripper command. In every layer, one attention spans both streams with shared projections, followed by separate feed-forward networks; world tokens use layer normalization, and action tokens use adaptive layer normalization conditioned on the flow time. With S, M, and A denoting the scene, summary, and action tokens, the attention mask allows $S \to S$, $M \to \{S, M\}$, and $A \to \{S, M, A\}$, where $X \to \mathcal{Y}$ means that tokens in X attend to tokens in $\mathcal{Y}$. The world stream never reads the actions. The action stream reads, in every layer, both the patch states shaped by scene-flow supervision and the summary states shaped by target-trajectory supervision.

Training objective. For a chunk $\mathbf{a}_1$ normalized with training statistics, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and flow time $u \sim \mathcal{U}[0, 1]$, the action stream v_ψ regresses the flow-matching velocity

$$\mathcal{L}_A = \mathbb{E}\,\mathrm{MSE}\big(v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t),\, \mathbf{a}_1 - \mathbf{a}_0\big), \quad (4)$$

where $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$, $\mathcal{S}_t$ denotes the inputs in Eq. (1), and the MSE averages over valid commands and action dimensions. The world and action streams are trained jointly with

$$\mathcal{L} = \mathcal{L}_A + \lambda_F \mathcal{L}_F + \lambda_P \mathcal{L}_P + \lambda_{\mathrm{aux}} \mathcal{L}_{\mathrm{aux}}, \quad (5)$$

where $\mathcal{L}_F$ and $\mathcal{L}_P$ are the flow and pose losses of Eqs. (2) and (3), and $\mathcal{L}_{\mathrm{aux}}$ sums auxiliary appearance losses.

Inference. At deployment, FoveaWAM skips the forecast heads, as recent world–action models do [21, 53, 56]. Because world tokens attend neither to the actions nor to the flow time, the world stream runs once per query, and the action stream integrates 5 Euler steps on the cached keys and values of the scene and summary tokens. Commands are indexed by time from capture, so the policy needs no latency input: the executor hands over each chunk at the first control-period boundary at or after inference completes and starts from the command indexed to that boundary. On the robot, the latency of a query also includes tracking the target in the newest frame. Further implementation details are in the supplementary material.

4. Experiments

We test whether role-aware precision and low latency together improve manipulation when the scene keeps moving during inference. We first compare FoveaWAM with existing policies, each charged with its own latency (Sec. 4.2), then analyze which parts of its design account for the results (Sec. 4.3), and finally verify it on a real robot with tracked target poses (Sec. 4.4).

4.1. Experimental Setup

OnTimeBench. Unlike the evaluators of DOMINO [14] and PhysMani-Bench [54], which advance the simulation only after a policy has returned its prediction, OnTimeBench shares one physical clock among observation, scene motion, and action execution, so the scene keeps moving while a policy computes. It contains fourteen tasks in five families, built in MuJoCo/robosuite [41, 62]: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact (Fig. 4). Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report all OnTimeBench results on 1,400 held-out scenes (100 per task).

Evaluation. During each query, physics advances by the method's inference latency, drawn from its latency distribution measured in its native deployment setting at batch size 1 on one RTX 5090, the GPU of our robot. All methods share one executor, which executes each chunk from the command meant for the control-period boundary at which the chunk arrives. Besides this native setting, we add 150 ms of latency to every method and, as a reference without the cost of computation, pause physics during inference. We report the task-balanced success rate (SR) and the p95 time-to-action, the measured time from an observation to its first command; paired 95% intervals come from a bootstrap over held-out scenes. The supplementary material gives the full evaluation protocol.

Baselines. We compare with the reactive policies ACT [59] and Diffusion Policy [12] and the VLA $\pi_{0.5}$ [5]; the dynamic VLAs DynamicVLA [47] and PUMA [14]; the WAM

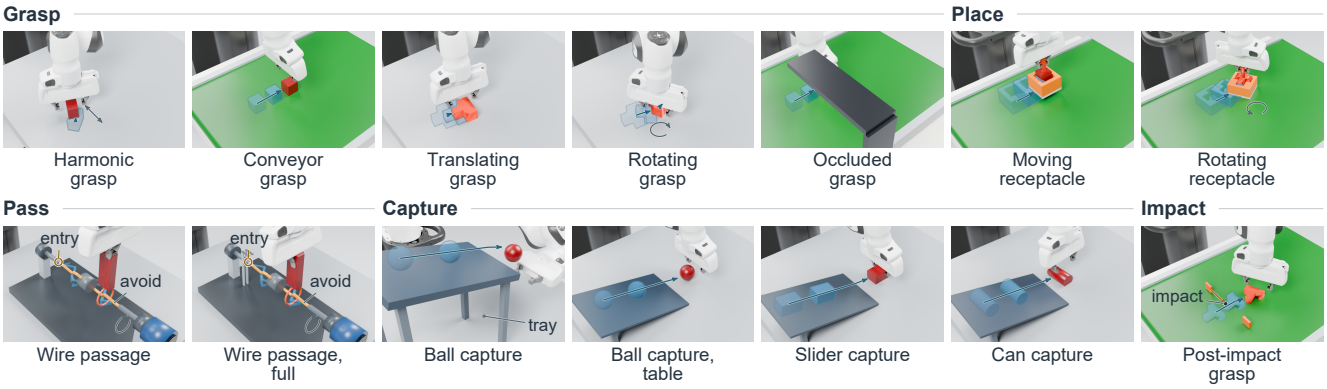


Figure 4. **OnTimeBench.** Fourteen tasks in five families, rendered from recorded simulator states with a common agent view. Translucent blue poses show selected history, fading with age; navy arrows follow recorded translational target motion. Local gray symbols denote rotation or oscillation qualitatively. An amber arrow shows the recorded striker approach; the *impact* label marks its contact location. Both Pass panels show passage: the wire is a target at entry and an obstacle along the path; Full additionally requires acquisition, exit and release. Occluded motion stays hidden. These native-expert examples illustrate task dynamics, not FoveaWAM outcomes; no future prediction is shown.

OpenWAM-α [43]; and the dynamic WAM DynamicWAM [27] (Table 1). Each baseline is trained on the same demonstrations with its official architecture and a task-adapted recipe. FoveaWAM receives a single agent-view RGB-D frame and a history of ground-truth SE(3) target poses from the simulator. To give baselines the same target-motion information, $\pi_{0.5}$ and DynamicWAM also run with our SE(3) history tokens (+SE(3)).

Implementation. The transformer of FoveaWAM has 12 layers of width 1024 with 16 heads and 271.8M trainable parameters on top of the frozen MAE ViT-B/16 encoder. On OnTimeBench, it trains from random initialization with AdamW for at most 20 epochs, and we keep the checkpoint with the lowest validation loss. The selected seed-0 run stops after 10 epochs (25,870 updates); its best checkpoint is from epoch 8 (20,696 updates). Evaluation uses compiled BF16 and five Euler steps with TF32 disabled. Compilation and recorded-observation warm-up finish before physics starts; history and the inference seed are then reset.

4.2. Dynamic Manipulation Results

Comparison with baselines. With its measured latency, compiled BF16 FoveaWAM reaches 76.0% SR on held-out OnTimeBench scenes (Table 1); the strongest baseline, PUMA, reaches 59.6%, 16.4 points lower (95% paired interval [14.3, 18.5]), and the strongest baseline receiving the same target-motion history, $\pi_{0.5}$+SE(3), reaches 52.8%. Its p95 time-to-action is 15.1 ms, compared with PUMA's 60.6 ms and OpenWAM-α's 151.7 ms.

Effect of inference delay. Charging computation changes the ranking. With physics paused during inference, FoveaWAM reaches 83.0% SR, 8.1 points above the strongest baseline in this setting, $\pi_{0.5}$+SE(3); with native latency, PUMA becomes the strongest baseline and the margin grows to 16.4 points, while the video WAMs, OpenWAM-α and DynamicWAM, fall from 58.3% and 49.1% to 46.1%

Table 1. **Results on OnTimeBench.** Task-balanced SR (%) on 1,400 held-out scenes with each method's native latency, with 150 ms added, and with physics paused during inference. p95: time-to-action in each method's native deployment setting on one RTX 5090. History: inputs that carry past motion beyond the current frame. OpenWAM-α uses its official caching and compilation, and (off) disables both. PUMA is timed with its official three-camera input; the other baselines with two cameras; FoveaWAM runs compiled BF16 inference on one RGB-D view and ground-truth target poses from the simulator. Best and second best per column are highlighted.

Method	History			SR (%) ↑			p95 (ms) ↓
	Wrist RGB	Flow	SE(3)	Native	+150 ms	Paused	
ACT [59]				41.6	24.3	40.6	**5.8**
DP [12]	✓			30.7	15.9	31.0	68.9
$\pi_{0.5}$ [5]				53.8	27.8	66.0	59.6
$\pi_{0.5}$+SE(3)			✓	52.8	28.2	74.9	61.2
DynamicVLA [47]	✓			50.9	23.4	54.6	97.3
PUMA [14]		✓		59.6	**31.1**	52.8	60.6
OpenWAM-α [43]				46.1	24.7	58.3	151.7
OpenWAM-α (off)				15.8	10.4	58.3	466.3
DynamicWAM [27]		✓		40.5	20.6	49.1	149.6
DynamicWAM+SE(3)		✓	✓	36.8	21.9	64.1	153.3
FoveaWAM			✓	**76.0**	25.6	**83.0**	15.1

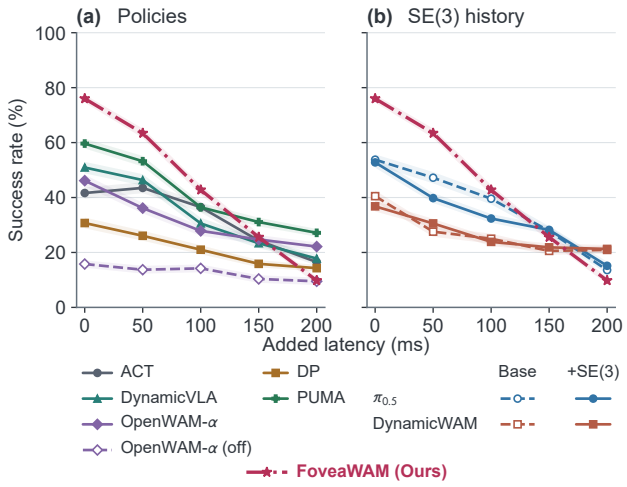


Figure 5. **Success under added latency.** Task-balanced SR on 1,400 held-out OnTimeBench scenes. Delay is added to each method's native latency. (a) Baseline deployments; off disables OpenWAM-α caching and compilation. (b) Base versus +SE(3)-history variants. PUMA is timed with its official three-camera input. FoveaWAM uses compiled BF16 (p95 15.1 ms). Bands: 95% task-stratified scene-bootstrap intervals.

and 40.5% (Table 1). With 150 ms added to its measured inference latency, FoveaWAM reaches 25.6% SR; the strongest baseline under this condition, PUMA, reaches 31.1%. Fig. 5 shows all five measured points from 0 to 200 ms of added latency.

Results on DOMINO. To test the design beyond OnTimeBench, we train FoveaWAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of DOMINO [14] with its demonstrations and evaluate them with its native evaluator. As on OnTimeBench, FoveaWAM reads ground-truth target poses from the simulator. FoveaWAM reaches an overall SR of 32.5%, 5.6 points above the strongest baseline, DynamicWAM (26.9%), and well above PUMA (20.2%) and $\pi_{0.5}$ (13.3%) (Table 2). Among the baselines, PUMA is strongest on Intercept tasks at 36.3%. More results are in the supplementary material.

4.3. Analysis of Role-Aware Modeling

Six controls each change one part of FoveaWAM and keep its data and recipe (Fig. 6). Except for the control without target history, each changes only training losses whose heads are skipped at deployment, so it shares the inference of FoveaWAM. We report each change in SR with native latency and with 150 ms added.

Allocation of precision. Without the dedicated SE(3) supervision, the target receives only the uniform coarse scene-flow supervision shared by the whole scene, and SR drops by [X.X] and [X.X] points. Making this uniform flow fine instead, on a [56×56] grid that places [about X] points on a typical target, changes SR by [X.X] and [X.X] points, [so precision spread over the whole scene does not replace a dedicated target trajectory]. Without the scene-flow supervision, the target keeps its SE(3) trajectory while the rest of the scene, including the context, loses its motion supervision,

Table 2. **DOMINO results** on 24 single-arm Franka clean-L1 tasks, with the same 2,400 test scenes per method (100 per task). Intercept/Track contain 8/16 tasks; All weights all 24 tasks equally. SR and MS use a 0–100 scale and retain failures. Best and second best are highlighted.

Method	Intercept	Track	All ↑	MS ↑
$\pi_{0.5}$	23.8	8.1	13.3	27.8
DynamicWAM	16.8	31.9	26.9	41.4
PUMA	**36.3**	12.2	20.2	35.4
FoveaWAM	18.6	**39.4**	**32.5**	**49.1**

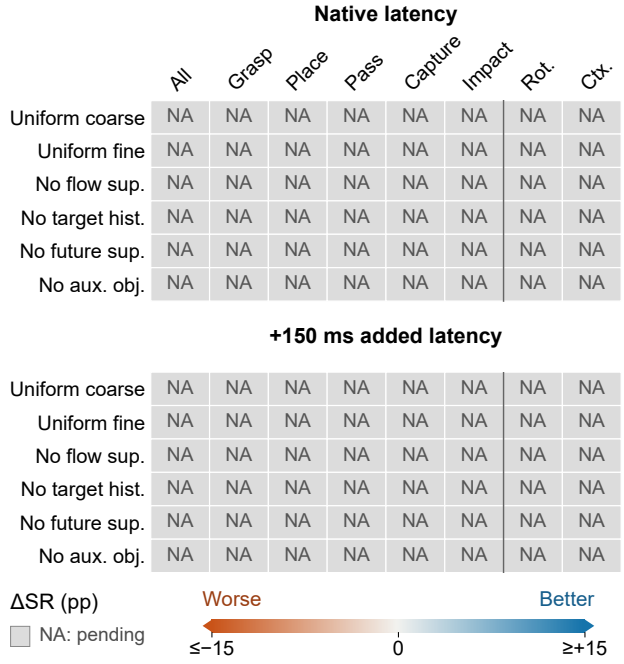


Figure 6. **Ablation effects by task (design preview; data pending).** ΔSR = SR(control) − SR(full), in percentage points; colors saturate at ±15. Rows: the six controls of Sec. 4.3; *uniform coarse flow* removes the dedicated SE(3) supervision, so that the target receives only the scene-flow supervision shared by the whole scene, and *uniform fine flow* also removes it but supervises the flow on a [56×56] grid. The full model reaches [XX.X]% SR with native latency and [XX.X]% with +150 ms. Rot./Ctx.: predeclared, overlapping rotation-sensitive/context-active groups (Table S2). Bold: the paired 95% scene-bootstrap interval excludes zero. NA: not available.

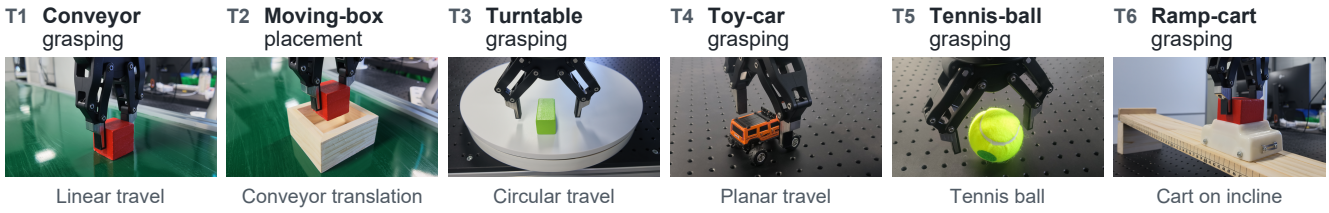


Figure 7. **Real-robot task setups.** Photographs of conveyor grasping (T1), placement into a moving box (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and block grasping from a ramp cart (T6).

Table 3. **Real-robot results.** Success rate (%) on the six tasks of Sec. 4.4, 20 trials per task and method; All pools the trials. p95: end-to-end time-to-action (ms) on one RTX 5090, including tracking for FoveaWAM. Best per column is highlighted, including ties.

Method	T1	T2	T3	T4	T5	T6	All ↑	p95 (ms) ↓
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
ACT	—	—	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—	—

and SR drops by [X.X] and [X.X] points. All three keep all auxiliary objectives, as the full model does. If precision follows the roles, the SE(3) supervision should matter most on the predeclared rotation-sensitive tasks, and the flow supervision on the context-active tasks, whose context constrains the robot or moves the target. [Whether Fig. 6 shows this pattern, and what it implies for the allocation of precision.]

Target motion history. Removing the past target poses with their velocity and acceleration features, while keeping the current image, the current target pose, and the robot history, lowers SR by [X.X] and [X.X] points. [The drop grows with the added latency,] as expected if the history supplies the target's velocity: a longer delay lets a velocity error accumulate further.

Future supervision and auxiliary objectives. Removing all future supervision, that is, the SE(3), scene-flow, and future appearance targets, while keeping the reconstruction of the current target, leaves inference unchanged yet lowers SR by [X.X] and [X.X] points, most on [tasks with non-uniform target motion]. Removing instead the auxiliary objectives $\mathcal{L}_{\text{aux}}$, including the current-target reconstruction, and keeping both geometric forecasts changes SR by [X.X] points, [what this implies for the role of the auxiliary objectives]. [One-sentence summary of which components matter, and where.]

4.4. Real-Robot Evaluation

Setup. We evaluate six tasks on a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera (Fig. 7): conveyor grasping (T1), placement into a moving box (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and block grasping from a ramp cart (T6). Baselines are the strongest simulation baseline, $\pi_{0.5}$; a low-latency imitation policy, ACT; and a dynamic WAM, DynamicWAM. All methods train or fine-tune on the same 100 teleoperated demonstrations per task, FoveaWAM from a 48-layer base pretrained on DROID [19], and run 20 trials per task with shared trial keys. Baselines receive RGB [and a wrist camera if their recipe uses one]; FoveaWAM receives the third-person RGB-D frame and FoundationPose [44] target estimates, updated at every control-period boundary before its query.

Results. FoveaWAM succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline], a paired difference of [X.X] points (95% interval [[X.X, X.X]]; Table 3). Its p95 latency, including tracking, is [XX] ms, [within one control period], whereas [name] needs [XX] ms. The margin is largest on [tasks]. [Whether the ranking of Table 1 carries over to real perception, and what tracking costs.] [One trajectory-verified contact example: how the target's pose, rotation, or context explains a success of FoveaWAM or a failure of a baseline.] The dominant failure of FoveaWAM is [failure mode]. Additional details are in the supplementary material.

5. Conclusion

FoveaWAM spends precision where contact depends on it: a metric SE(3) history and forecast for the target and a sparse 3D flow for the whole scene, with the forecasts used only as training supervision. When every method is charged its own latency, these choices let FoveaWAM act both precisely and quickly: it outperforms reactive policies, VLAs, and world–action models, including ones built for moving targets, on OnTimeBench and on six real-robot tasks at a p95 latency of 15.1 ms on OnTimeBench with compiled BF16 inference, and it also leads on DOMINO. The strongest baseline is

PUMA under native latency and $\pi_{0.5}$+SE(3) in the paused column. [The one or two strongest ablation findings.] We will release OnTimeBench so that future dynamic policies are evaluated with their latency charged.

Limitations. Deployment requires a specified rigid target and reliable online pose estimates. The target forecast predicts one future per horizon and ignores the effects of robot contact on target motion. Uncertainty-aware forecasts and contact-conditioned dynamics could address these limits. Independently moving context lies outside our task set and would need its own motion history, for example from scene flow.

LLM Usage Statement

We used OpenAI Codex to assist with literature review, code implementation and debugging, experiment execution, and language editing of the manuscript. Codex also assisted in developing plotting code and refining figure and manuscript layouts. The authors reviewed AI-assisted content, checked references against original sources, and validated the implementation through testing and experiments. The authors take full responsibility for the final manuscript, experimental results, and conclusions.

References

[1] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[2] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.

[3] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[4] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 3

[5] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 1, 5, 6, 15

[6] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1

[7] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2, 14

[8] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, pages 173–180, 2017. 3

[9] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 3

[10] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[11] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Joon-Young Lee, and Alexander Schwing. Putting the object back into video object segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 17

[12] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 5, 6, 15

[13] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 15873–15879, 2025. 3

[14] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 1, 2, 3, 5, 6, 7, 15, 16

[15] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *IEEE Robotics and Automation Letters*, 11(3):2466–2473, 2026. 3

[16] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 3, 13

[17] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 2, 3

[18] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 3, 13

[19] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 8, 13

[20] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713, 2025. 1

[21] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 3, 5

[22] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[23] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 5

[24] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang,

Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 17

[25] Yuejiang Liu, Jubayer Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, pages 4594–4627, 2025. 2

[26] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 3

[27] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 6, 15

[28] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 2

[29] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *Robotics Research: The 19th International Symposium ISRR*, pages 132–157. Springer, 2022. 3

[30] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[31] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, pages 28085–28128, 2025. 17

[32] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799, 2023. 3

[33] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[34] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 13

[35] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[36] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. In *European Conference on Computer Vision*, pages 437–453, 2026. 3

[37] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 115917–115933, 2026. 3

[38] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. In *Conference on Robot Learning*, 2026. Accepted for publication. 1, 2

[39] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 3

[40] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, pages 92033–92052, 2025. 3

[41] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5, 14

[42] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. In *Advances in Neural Information Processing Systems*, pages 35928–35959, 2025. 1

[43] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 3, 6, 15

[44] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 8, 17

[45] Chuan Wen, Xingyu Lin, John Ian Reyes So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[46] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, pages 10641–10662, 2024. 3

[47] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 5, 6, 15

[48] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the

cross-domain manipulation interface. In *Conference on Robot Learning*, pages 2475–2499, 2025. 3

[49] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2, 3

[50] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 3

[51] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2025. 3

[52] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 3

[53] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 3, 5

[54] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. In *European Conference on Computer Vision*, pages 20–39, 2026. 2, 3, 5

[55] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[56] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 3, 5

[57] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[58] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 3

[59] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 5, 6, 15

[60] Ruijie Zheng, Jing Wang, Scott Reed, et al. FLARE: Robot learning with implicit world modeling. In *Proceedings of The 9th Conference on Robot Learning*, pages 3952–3971. PMLR, 2025. 3

[61] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 3

[62] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5, 14

[63] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183, 2023. 1

A. Method and Training Details

A.1. Architecture and Training

Table S1 lists the hyperparameters of FoveaWAM on OnTimeBench.

Table S1. Hyperparameters of FoveaWAM on OnTimeBench. A training window holds 8 consecutive query positions, each with its own causal history.

Parameter	Value
Model	
Input image	224×224 RGB-D, one view
Visual encoder	MAE ViT-B/16, frozen
History length	8 steps (350 ms)
Forecast horizons	8 (50 ms–400 ms)
Summary tokens	16
Layers	12
Width	1024
Heads	16
Feed-forward width	4096
Trainable parameters	271.8M
Action	
Action dimension	7
Chunk length	8
Euler steps	5
Noise samples per query	8
Training	
Optimizer	AdamW
Betas	$(0.9,\ 0.999)$
Weight decay	10^{-4}
Learning rate	2×10^{-4}
LR schedule	cosine, 5,174 warm-up updates
Batch size	512 windows
Epochs	10 (cap: 20)
Updates / selected checkpoint	25,870 / 20,696
Gradient clipping	0.1
$\lambda_F, \lambda_P, \lambda_R, \lambda_{\text{aux}}$	0.1
Training precision	FP32, frozen encoder BF16
Hardware	64 RTX PRO 6000 GPUs, [X] h

Scene tokens. Each patch token of the frozen MAE ViT-B/16 [16] adds a learned view embedding and linear embeddings of its mean valid metric depth and of the base-frame position $\mathbf{x}_i$ of its back-projected center pixel; a patch without valid depth at its center receives no position embedding. Only the patch queries and keys receive 3D rotary embeddings from $\mathbf{x}_i$, and the same points are the source points of the sparse 3D flow.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities and accelerations from finite differences over the elapsed times, and slot 7 holds the current pose in the base frame. Each dimension is standardized with training statistics, invalid dimensions are zeroed, and three validity flags (pose, interval, acceleration) are appended; a slot without any valid feature is masked out of attention. The robot tokens hold the end-effector position, rotation vector, and gripper opening at the same offsets, and 16 learned embeddings form the summary tokens.

Heads and losses. Each head is a linear layer on the final world-stream states, except the target reconstruction, a two-layer MLP. The pose head reads the mean summary state ($\mathcal{L}_P$, Eq. 3). The flow head gives each patch point's displacement and visibility logit at every horizon ($\mathcal{L}_F$, Eq. 2, with scale $s = 0.715$ m and visibility weight 1); valid but invisible points still supervise their displacement. Flow labels follow the rendered surface point at each patch center through the ground-truth rigid motion of its MuJoCo geometry; a future point is visible if it projects into the image onto the same geometry within a depth tolerance of $\max(1\,\text{mm},\ 0.001z)$. $\mathcal{L}_{\text{aux}}$ sums four losses: squared errors on future metric depth (valid pixels), on the RGB of surfaces that move by more than 1 mm, and on layer-normalized DINOv3 features [34], and an absolute error on the current target's RGB region reconstructed from the mean summary state.

Action stream. Each action token embeds a seven-dimensional command: a base-frame translation increment, a left-relative rotation vector, and a gripper command, relative to the measured end-effector pose at application. Command j of query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution (Eq. (S1)), and commands are standardized over the training split. At deployment, the gripper changes state only when its command leaves a deadband of 0.5 in Ball capture and 0.1 in every other task, including Ball capture, table.

A.2. Pretraining on DROID and Fine-Tuning

The real-robot base trains on DROID [19] (15 Hz) with the first exterior camera and its depth, using the 3D point tracks and calibrations released by PointWorld [18]. Its 42,087 usable episodes yield 598,889 anchor windows, 60,149 of them for validation; each anchor expands into 8 query positions as on OnTimeBench, and only the anchor carries flow and DINOv3 labels. Each patch follows one PointWorld track that starts inside it, preferring the one with the largest consistent motion so that small moving objects are covered; 47% of the point–horizon slots carry a label.

DROID provides no object poses, so the base has no target-state tokens or pose head; it trains the action loss, the sparse 3D flow with $\lambda_F = 1$ and no visibility term, and the future DINOv3 features with weight 0.1. It has 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates) at a global batch of 512 on 64 GPUs

with the optimizer and precision of Table S1. Each real-robot policy is fine-tuned from it on its task's demonstrations [updates, learning rate, and batch size], with the target-state tokens and missing heads freshly initialized; no real-robot policy uses simulation data or weights.

B. OnTimeBench and Evaluation Protocol

B.1. Benchmark Design

The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [41, 62] with a Franka Panda arm and a parallel gripper under robosuite's operational-space pose controller, with a control period of 50 ms and a physics step of 2 ms. A fixed third-person camera with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector translation and rotation-vector increment in the base frame with a gripper command, clipped to 0.10 m per translation axis and 0.5 rad per rotation component. Table S2 lists the fourteen tasks.

B.2. Scenarios, Demonstrations, and Splits

Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges. Table S2 summarizes target motion. Other sampled parameters include harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, and ramp slopes of 5–15°. In an occluded scene, the target is hidden for 0.5–3.0 s and then reappears. Only physically feasible scenes are kept, and evaluation replays the stored scenes.

The expert is a task-specific closed-loop state machine with privileged simulator state that anticipates the target's motion, for example by predicting the interception point in the capture tasks. A demonstration is kept if the expert succeeds, a replay of its actions reproduces the episode, and its path and jerk pass quality checks. The training pool holds 11,200 demonstrations, 800 per task, recorded at 20 Hz with RGB-D, robot states, commands, and simulator ground truth, and 350 further episodes form the validation split; policies receive only the inputs of Sec. 3.

We select configurations on 1,400 development scenes (100 per task) and report all OnTimeBench results on 1,400 held-out scenes shared by all methods, generated with the same ranges from seeds used in no training, validation, or development episode.

B.3. Real-Time Evaluation and Metrics

Executor. All methods share one executor on the 50 ms control clock. Command j of query t is meant for $\tau_t + j\Delta_a$, and its execution time (Sec. 3.1) is

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}, \tag{S1}$$

where $d_{\mathrm{task}} = 50$ ms for the post-impact grasp, which applies each command 50 ms after it is issued, and $d_{\mathrm{task}} = 0$ otherwise. As in RTC [7], latency counts in whole control periods: a query with total latency L_t hands over its chunk at the first boundary at or after it ends, $k_t = \lceil L_t/\Delta_a \rceil$ periods after capture, and execution continues from command k_t, the one meant for that boundary. The policy then observes again and starts the next query; meanwhile the previous chunk continues, so a policy replans as often as its latency allows. A chunk holds the method's native number H_m of commands (Table S3) and is dropped if $k_t \geq H_m$. Before the first chunk arrives and whenever a chunk runs out, the controller holds the current arm target and gripper command; no command is repeated. The executor runs asynchronously, as in the streaming of DynamicVLA; no method, including FoveaWAM, inpaints across chunks as in RTC.

Protocols. In real time, physics advances during each query by the method's latency, drawn per query from its distribution measured on one RTX 5090, while the rollouts themselves run on RTX PRO 6000. With paused physics, inference takes no physics time and a policy replans every 50 ms; this removes the wait for inference but also raises the feedback rate, so it is a reference rather than an upper bound. The latency sweep adds 0 to 200 ms in steps of 50 ms after inference in real time.

Latency measurement. We time each method alone on one RTX 5090 at batch size 1 in each baseline's release precision and in compiled BF16 for FoveaWAM, over three rounds of 30 warm-up and 300 measured calls on recorded observations with the required causal history. A timed query starts from the newest observation and ends when its first command is available. It includes all computation that needs the newest frame, such as preprocessing, image encoding, the online flow and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. The forecast heads of FoveaWAM are skipped at deployment and excluded from time-to-action. A query misses its deadline when this time-to-action exceeds 50 ms, and the action age of a command is its execution time minus the capture time. Appendix C.2 reports the measured values.

Metrics. SR weights the 14 tasks equally. The position error at first contact e_p is the distance of the end effector from the expert's contact frame on the target; in the placement tasks, which start with the object held, it uses the contact of the held object with the receptacle and the expert's placement pose. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact remain in the SR denominator. Paired 95% intervals bootstrap held-out scenes within each task, jointly across methods.

Table S2. **The fourteen OnTimeBench tasks.** The target is the body whose pose the policy receives (Sec. 3.1). For placement it is the receptacle. For passage it is the wire entry, and the rest of the wire is context. Motion gives linear speed in m/s or the wire's angular speed in degrees per second. For each capture task, we estimate speed from the two samples 50 ms apart that bracket the first recorded ramp exit in each of 100 held-out fixed-robot reference trajectories. A grasp requires bilateral fingerpad contact, a lift of at least 0.05 m, and a continuous 0.5 s hold. An episode fails at unsafe contact, leaving the workspace, a drop, or the time limit. In the group column (Fig. 6), R marks tasks where the target rotates during the lead and its orientation matters at contact. C marks tasks with an obstacle clearance or a driver such as a conveyor, ramp, or striker.

Task	Code	Target	Motion range	Success	Limit (s)	Group
Harmonic grasp	`grasp`	block	0.03–0.20	grasp	8	R
Conveyor grasp	`t1_belt`	part	0.04–0.10	grasp	8	C
Translating grasp	`t1_xyz`	part	0.04–0.15	grasp	8	–
Rotating grasp	`t1_so3`	part	0.04–0.15	grasp	8	R
Occluded grasp	`t1_occ`	part	0.04–0.10	grasp without canopy contact	8	C
Moving receptacle	`t2_trans`	receptacle	0.03–0.06	held object at rest inside	10	–
Rotating receptacle	`t2_se3`	T-shaped receptacle	0.03–0.06	held T-part at rest inside	10	R
Wire passage	`t3_phase`	wire entry	2.5–7.5°/s	align within 30 mm, pass without contact	21	R, C
Full wire passage	`t3_full`	wire entry	2.5–7.5°/s	grasp wand, pass, exit, release	32	R, C
Ball capture	`t4_sphere`	ball	0.67–1.14	grasp before the tray	6	C
Ball capture on table	`t4_sphere_tabletop`	ball	0.59–1.03	grasp while rolling	6	C
Slider capture	`t4_slider`	slider	0.55–1.12	grasp while sliding	6	C
Can capture	`t4_can`	can	0.59–0.97	grasp before tipping	6	R, C
Post-impact grasp	`t5_replan`	part	0.12–0.14	grasp along its axis	6	R, C

Table S3. **Baseline training.** The training budget gives optimizer updates and global batch size. H is the number of commands returned per query. The parameter counts include trainable parameters only. DOM ckpt. denotes the released DynamicVLA checkpoint trained on DOM.

Method	Initialization	Updates×batch	H	Params
ACT [59]	scratch	50,000 × 32	100	83.9M
DP [12]	scratch	50,000 × 32	15	288.7M
$\pi_{0.5}$ [5]	`pi05_base`	30,000 × 256	10	3,353.4M
$\pi_{0.5}$+SE(3)	`pi05_base`	30,000 × 256	10	3,357.7M
DynamicVLA [47]	DOM ckpt.	50,000 × 44	20	99.1M
PUMA [14]	official	40,000 × 64	16	4,516.5M
OpenWAM-α [43]	official	34,490 × 192	32	6,021.2M
DynamicWAM [27]	official	40,000 × 128	16	988.8M
DynamicWAM+SE(3)	official	40,000 × 128	16	989.4M
FoveaWAM	scratch	25,870 × 512	8	271.8M

B.4. Baselines

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe (Table S3), trains on the same 11,200 demonstrations mapped to its action space through the same Cartesian controller, and is configured on development scenes. Rollouts use the agent view each baseline was trained with, any wrist slot masked, and the histories of Table 1; masking the wrist view changes p95 latency by at most 3.8 ms in the seven configurations timed both ways. DP uses its authors' real-robot DDIM sampler with 16 steps, and OpenWAM-α runs with video decoding disabled and is timed with and without its official cache and compilation.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags. A two-layer MLP with a zero-initialized output layer maps them to the model's width; for $\pi_{0.5}$ they follow the image tokens in the prefix, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. Training follows the base model's recipe.

C. Additional Simulation Results

C.1. Per-Task Results

Table S4 uses the Native real-time protocol with +0 ms extra delay. Every method fails on rotating-receptacle placement. FoveaWAM reaches 38% on moving receptacle placement versus DynamicVLA's 39%, and 10% on full wire passage versus DynamicVLA's 12%; both paired intervals include zero. It ties the strongest baseline on phase-only wire passage, tabletop ball capture and can capture. The largest recorded failure classes for FoveaWAM are rim impulse violations (160 of 1,400 scenes) and wire contacts (55 scenes).

C.2. Latency

Table S5 lists all eleven configurations at five added latencies; Fig. 5 plots all 55 accepted points.

Table S4. **Per-task held-out success** (%) on OnTimeBench for every method of Table 1, Native real-time protocol: each method's measured inference latency plus **+0 ms extra delay**, with physics continuing during inference; 100 paired scenes per task. FoveaWAM uses compiled BF16 C (p95 15.1 ms). DP: Diffusion Policy. DVLA: DynamicVLA. OWAM: OpenWAM-α with official cache/compile; OW-off: both disabled. DWAM: DynamicWAM. +: with our SE(3) history tokens. Δ: FoveaWAM minus the strongest other method on that task; ties follow table order. The task-balanced row compares against the strongest overall baseline, PUMA. Brackets give pointwise 95% paired scene-bootstrap intervals, conditional on the selected comparator and checkpoint (100,000 resamples; no selection adjustment). The last two rows give first-contact position error e_p (mm), averaged over the 13 contact tasks, and the number n of episodes with a contact (Appendix B.3); –: no complete 13-task mean. Δ shading is blue for higher SR and orange for lower, saturating at 15 points. Blue/bold and light blue mark the best and second-best displayed values.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	OWAM	OW-off	DWAM	DWAM+	**Ours**	Δ [95% CI]
Harmonic grasp	49.0	31.0	51.0	85.0	39.0	85.0	41.0	1.0	50.0	51.0	**95.0**	+10.0 [3.0, 17.0]
Conveyor grasp	45.0	45.0	96.0	58.0	89.0	89.0	62.0	0.0	59.0	47.0	**100.0**	+4.0 [1.0, 8.0]
Translating grasp	14.0	33.0	59.0	38.0	61.0	51.0	53.0	0.0	53.0	15.0	**89.0**	+28.0 [18.0, 38.0]
Rotating grasp	24.0	20.0	32.0	44.0	26.0	67.0	42.0	3.0	30.0	37.0	**92.0**	+25.0 [14.0, 36.0]
Occluded grasp	30.0	28.0	37.0	51.0	80.0	84.0	34.0	0.0	44.0	36.0	**97.0**	+13.0 [6.0, 21.0]
Moving receptacle	26.0	8.0	26.0	19.0	**39.0**	0.0	28.0	0.0	0.0	0.0	38.0	-1.0 [-16.0, 14.0]
Rotating receptacle	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	+0.0 [0.0, 0.0]
Wire passage	99.0	99.0	**100.0**	**100.0**	99.0	87.0	50.0	58.0	23.0	23.0	**100.0**	+0.0 [0.0, 0.0]
Wire passage, full	0.0	3.0	1.0	5.0	**12.0**	0.0	0.0	0.0	0.0	0.0	10.0	-2.0 [-10.0, 6.0]
Ball capture	36.0	16.0	46.0	21.0	41.0	39.0	45.0	13.0	24.0	23.0	**76.0**	+30.0 [20.0, 41.0]
Ball capture, table	71.0	81.0	**100.0**	**100.0**	88.0	**100.0**	99.0	88.0	98.0	**100.0**	**100.0**	+0.0 [0.0, 0.0]
Slider capture	56.0	30.0	58.0	68.0	28.0	76.0	71.0	31.0	47.0	63.0	**80.0**	+4.0 [-6.0, 14.0]
Can capture	91.0	31.0	98.0	99.0	72.0	**100.0**	**100.0**	5.0	96.0	**100.0**	**100.0**	+0.0 [0.0, 0.0]
Post-impact grasp	42.0	5.0	49.0	51.0	39.0	57.0	21.0	22.0	43.0	20.0	**87.0**	+30.0 [20.0, 41.0]
Task-balanced	41.6	30.7	53.8	52.8	50.9	59.6	46.1	15.8	40.5	36.8	**76.0**	+16.4 [14.3, 18.5]
e_p (mm)	32.2	36.2	31.2	32.5	30.0	27.2	31.7	–	31.2	37.0	**25.2**	–
n	1110	941	1231	1187	1203	1272	1115	439	1193	970	1295	–

Table S5. **Success versus added latency** on 1,400 held-out OnTimeBench scenes per point (SR in %, real-time protocol). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1. Blue/bold and light blue mark the best and second-best values per column.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	41.6	43.5	36.5	24.3	16.5
Diffusion Policy	30.7	26.1	21.0	15.9	14.3
$\pi_{0.5}$	53.8	47.2	39.6	27.8	13.6
$\pi_{0.5}$ + SE(3)	52.8	39.8	32.4	28.2	15.1
DynamicVLA	50.9	46.4	30.6	23.4	17.8
PUMA	59.6	53.2	36.4	**31.1**	**27.1**
OpenWAM-α	46.1	36.1	27.9	24.7	22.1
OpenWAM-α (off)	15.8	13.7	14.2	10.4	9.5
DynamicWAM	40.5	27.6	25.0	20.6	20.9
DynamicWAM + SE(3)	36.8	30.6	23.9	21.9	21.3
FoveaWAM	**76.0**	**63.4**	**42.8**	25.6	9.8

Timing. Table S6 breaks one query of FoveaWAM down by stage.

Table S6. **Latency breakdown** of one query of compiled BF16 FoveaWAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot. The policy total uses 900 measured queries and excludes pose tracking; magenta dashes mark pending measurements.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Observation encoding + H2D	4.83	5.28
World stream (once per query)	3.93	4.22
Action stream (5 Euler steps)	4.27	4.41
Policy time-to-action (total)	14.66	15.65
Training-only readouts (for reference)	0.37	0.41
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

C.3. Results on DOMINO

We use the 24 single-arm Franka tasks of the clean level-1 setting of DOMINO [14] with its demonstrations, scenes, and native evaluator. The manipulation score (MS) scales route completion by penalties for leaving the workspace or view and for collisions. FoveaWAM reads the ground-truth pose of the instructed target from the simulator and masks slots without a valid pose (Appendix A.1). The RGB-based

baselines do not receive this pose.

Data and adaptation. The data split has 1,200 training and 240 validation demonstrations. Policies execute absolute joint-position targets with DOMINO's native controller and adapt their state and action projections to its eight-dimensional joint space. Normalizers use training data only. The official sampler of DynamicWAM skips demonstrations too short for its windows, which leaves it 1,054 training and 210 validation demonstrations and none for two tasks; all 24 tasks remain in the test set. DynamicWAM trains for 40,000 updates at global batch 128. $\pi_{0.5}$ starts from its official base, trains for 30,000 updates at global batch 256, and is evaluated with its final EMA. PUMA keeps its published weights and frozen teacher and trains at global batch 64 with early stopping on validation action MSE, which selected update 6,000. For development, FoveaWAM trains from scratch with a global batch of 512 and stops after 1,575 updates. The lowest validation loss selects the checkpoint at update 1,365. Development rollouts select three executed actions per query.

C.4. Ablation Controls

Each of the six controls of Sec. 4.3 (Fig. 6) changes one component of the configuration of Table S1 and keeps the data, the recipe with its checkpoint selection, and the executor. Removing a loss also removes its head, and the control without target history removes target-state slots 0–6 and keeps slot 7, the current pose (Appendix A.1). The uniform fine flow control removes $\mathcal{L}_P$ and the pose head and supervises the flow at [4×4] points in each patch, a [56×56] grid, with labels from the same ground-truth rigid motion.

D. Real-Robot Experiments

D.1. Platform and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]; a fixed third-person camera records RGB-D at [resolution and frame rate], calibrated by [procedure] before each session; baselines whose recipes use a wrist view also receive [wrist camera]. The task devices are installed on this platform one at a time (Fig. 7; Table S7).

D.2. Pose Tracking and Labels

Tracking is marker-free and uses one metric mesh per object instance; a ball is defined by its center and radius. In the first frame, Grounding DINO [24] localizes the object named by the task, SAM 2.1 [31] segments it, and FoundationPose [44] registers its pose; afterwards, Cutie [11] propagates the mask and FoundationPose refines a Kalman-filter prediction at each update. An empty mask makes the estimate invalid rather than carrying the previous pose forward, and [re-initialization after a loss]. The tracker updates on the newest frame at every control-period boundary and keeps that frame's capture time, from which the target-state tokens take their finite differences.

The target-state inputs of a demonstration replay the same tracker causally over its recorded RGB-D. The pose labels are the tracked poses at future boundaries, [refined offline over the episode], and the flow labels follow [3D point tracks of the recorded RGB-D, as for DROID (Appendix A.2)]. For the ball, the target-state tokens and the pose head use translation only. Over all test trials of FoveaWAM, the estimate was invalid at [X]% of control-period boundaries, with a longest gap of [X] s and [X] re-initializations[, most of them on task].

D.3. Training and Evaluation Protocol

The baselines keep the initialization, recipe, and chunk length of Table S3, and FoveaWAM is fine-tuned as in Appendix A.2; DynamicWAM executes [with or without real-time chunking]. Demonstrations, development trials, and test trials come from separate sessions.

Each task has 20 pre-generated trial keys that fix the object instance, its initial pose, and the device setting. For each key, the four methods run consecutively in a random order, so that drift of the apparatus does not align with the methods, after an operator resets the scene against a reference image. A trial ends at success, failure, a safety stop, or a time limit of about three times the task's median demonstration duration. A key is repeated only if the device deviates from it on its own ([speed outside ±X% of the setting or a device fault]); everything else, including a tracking loss of FoveaWAM, counts as a failure. Table S6 also gives the action age and the missed deadlines on the robot.

D.4. Results and Failure Analysis

Table S8. **Real-robot success with intervals.** All: success rate over the 120 test trials of each method (%) with its 95% Wilson interval. Δ: paired difference from FoveaWAM (points), with a bootstrap interval over trial keys within each task. Per-task rates are in Table 3.

Method	All [95% CI]	Δ [95% CI]
$\pi_{0.5}$	—	—
DynamicWAM	—	—
ACT	—	—
FoveaWAM	—	–

Table S8 gives the pooled success rates with their intervals, and Fig. S1 the failures. The failure categories, fixed before the test trials, are observable events: late (the gripper closes behind the target), misaligned grasp, slip or drop after contact, collision or safety stop, timeout, and tracking lost

Table S7. **Real-robot tasks.** Target: the tracked object. Trial key: what each of the 20 test keys fixes. Speed: target speed from the recorded RGB-D of all test trials (m/s, mean ± s.d.). Grasp: the object held between both fingers and lifted by [X] cm. Each task has 100 training and 20 validation demonstrations.

Task	Target	Motion	Trial key	Speed	Success	Limit (s)
T1	block	belt conveyor	[placement on the belt, belt speed]	—	grasp	[X]
T2	open box	belt conveyor	[box placement, belt speed]	—	held block stays in box	[X]
T3	block, unequal sides	turntable	[radius, initial angle and yaw, angular speed]	—	grasp across short sides	[X]
T4	toy car	remote driving, fixed throttle	[start pose, heading]	—	grasp	[X]
T5	tennis ball	[mechanism]	[release position, motion setting]	—	grasp	[X]
T6	block on cart	inclined rail	[slope, release point]	—	grasp	[X]

Figure placeholder: Real-robot failures

(a) Failures by category (≈35% width): one horizontal stacked bar per method ($\pi_{0.5}$, DynamicWAM, ACT, FoveaWAM), length = failed test trials of 120, segments in the fixed order late, misaligned, slip/drop, collision or safety stop, timeout, tracking lost. **(b) One paired trial key** (≈65%): two rows of five unretouched third-person frames, FoveaWAM (success) above and [baseline] (late) below, at the same times after the first command; gripper closure marked.

Figure S1. **Real-robot failures.** (a) Failures by category over the 120 test trials of each method. (b) One trial key of [task]: FoveaWAM grasps the target, whereas [baseline] closes its gripper behind it (late). The key is one on which the baseline fails in its most frequent category; frames are taken at the same times after the first command.

(FoveaWAM only). [The dominant failure of each method, and what it shows about latency; the most frequent failure of FoveaWAM.]